

Московский государственный университет имени М. В. Ломоносова

Факультет Вычислительной Математики и Кибернетики
Кафедра Математических Методов Прогнозирования

Квантизация диффузионных моделей машинного обучения

Курсовая работа

Выполнил:
студент 317 группы
Подъяблонский Егор Петрович

Научный руководитель:
к. к. н.,
Аланов Айбек

Москва, 2026

Содержание

1 Введение **2**

2 Основные понятия и метрики **2**

3 Рассматриваемые решения **5**
3.1 Равномерная скалярная PTQ 5
3.2 PTQ4DM 6
3.3 Q-Diffusion 6
3.4 BitsFusion 7
3.5 Vanilla VQ 7
3.6 AQUATIC-Diff 8
3.7 VQDM 9

4 Условия экспериментов **9**

5 Результаты запуска базовых методов **11**
5.1 Скалярные базовые методы 11
5.2 Векторные базовые методы на 16 слоях 11
5.3 Векторные базовые методы на 18 слоях 12
5.4 Лучшие базовые методы на 18 слоях 13

6 Предлагаемый RVQ-подход **13**
6.1 Постановка задачи RVQ 13
6.2 Activation-aware функционал 14
6.3 Ортогональное преобразование 15
6.4 Предварительные эксперименты с Hadamard 16
6.5 Влияние размера кодбука 17
6.6 Рекодинг после RVQ на полной матрице $X^\top X$ 17
6.7 Проверка увеличенной размерности фрагмента 19
6.8 Распределение размера кодбуков по RVQ-стадиям 19
6.9 Локальный рекодинг после RVQ-стадий 21

7 Итоговые выводы **21**

Список литературы **23**

1. Введение

В последние годы нейронные сети стали одним из основных инструментов машинного обучения. Они применяются в задачах обработки изображений, текста, речи, видео и других типов данных. Отдельное место среди современных архитектур занимают генеративные подходы, способные создавать новые объекты, похожие на элементы обучающей выборки. В частности, диффузионные модели показали высокое качество в задачах генерации изображений по текстовому описанию и стали одним из наиболее заметных классов таких методов.

Несмотря на высокое качество, практическое использование таких систем ограничено их вычислительной стоимостью. Современные диффузионные модели содержат большое число параметров, занимают значительный объем видеопамяти и часто предполагают наличие производительных графических ускорителей. Это затрудняет их локальное развертывание на пользовательских устройствах и ограничивает доступность подобных решений в условиях, где невозможно использовать несколько GPU или облачную инфраструктуру.

Масштаб проблемы можно оценить на примере Stable Diffusion 1.5, рассматриваемой в данной работе. Эта модель использует U-Net порядка 860 млн параметров и текстовый кодировщик порядка 123 млн параметров. Если в базовой оценке считать, что каждый параметр хранится в формате FP32, то одно число занимает 32 бита, то есть 4 байта. Тогда только U-Net занимает около

$$860 \cdot 10^6 \cdot 4 \approx 3.44 \cdot 10^9$$

байт памяти, а текстовый кодировщик добавляет еще примерно

$$123 \cdot 10^6 \cdot 4 \approx 0.49 \cdot 10^9$$

байт. В сумме эти две части содержат около 983 млн параметров и занимают порядка 3.93 ГБ только для хранения весов в FP32. С учетом остальных компонентов, промежуточных активаций и накладных расходов фреймворка фактический расход памяти становится еще выше.

Одним из способов снизить вычислительные требования и объем памяти является квантизация. Под квантизацией понимается замена исходного представления параметров на более компактный формат с меньшим числом бит на параметр. Такая процедура позволяет уменьшить стоимость хранения и потенциально ускорить инференс. При этом основной сложностью остается сохранение качества: слишком агрессивное сжатие может привести к заметной деградации результатов генерации.

Целью данной работы является изучение методов квантизации диффузионных моделей машинного обучения. В рамках исследования рассматриваются существующие подходы к решению этой задачи, проводится их экспериментальное сравнение и предлагается собственный способ уменьшения размера модели при сохранении качества генерации.

2. Основные понятия и метрики

Квантизация модели заключается в замене исходных чисел, которыми представлены параметры или промежуточные значения нейронной сети, на элементы из некоторого конечного множества. В простейшем случае каждому числу ставится в соответствие ближайшее значение из заранее заданной сетки. За счет этого уменьшается число бит, необходимое для хранения параметров, а при наличии аппаратной поддержки может снижаться и стоимость вычислений.

Принято различать скалярную и векторную квантизацию. При скалярной квантизации каждый вес обрабатывается независимо:

$$w \mapsto q(w), \qquad q(w) \in \mathcal{Q},$$

где $\mathcal{Q}$ - конечный набор допустимых значений. Такой подход прост в реализации и часто используется как базовый метод сжатия. Однако он не учитывает структуру групп весов и потому может быть недостаточно гибким при низкой битности.

Векторная квантизация работает не с отдельными числами, а с векторами. Матрица весов разбивается на фрагменты $v_i \in \mathbb{R}^d$, после чего каждый фрагмент заменяется ближайшим кодовым словом из кодбука:

$$v_i \mapsto c_{k(i)}, \qquad c_{k(i)} \in \mathcal{C}.$$

Здесь $\mathcal{C} = \{c_1, \ldots, c_K\}$ - набор обучаемых центроидов, а $k(i)$ - индекс выбранного кодового слова. В этом случае вместо самих векторов достаточно хранить кодбук и индексы. Такой подход потенциально лучше использует зависимости между соседними весами, но требует обучения кодбуков и аккуратного выбора размера вектора, числа кодовых слов и числа кодбуков.

Квантизацию можно применять к разным объектам модели. Чаще всего квантуются веса нейронной сети, так как именно они занимают постоянный объем памяти при хранении модели. В некоторых подходах дополнительно квантуются активации, то есть промежуточные тензоры, возникающие во время прямого прохода. В данной работе активации используются прежде всего для калибровки: они позволяют оценить, какие ошибки в весах сильнее влияют на выход слоя.

Наиболее прямой способ обучать квантизованное представление весов состоит в минимизации ошибки восстановления самих весов:

$$\|W - \widehat{W}\|_F^2,$$

где W - исходная матрица весов, а $\widehat{W}$ - ее квантизованное приближение. Такой критерий называется weight-only или функционалом восстановления весов. Он прост и не требует данных, но не учитывает, как слой используется в реальной модели. Ошибка в редко используемом направлении входного пространства может быть менее важной, чем такая же по норме ошибка в направлении, часто встречающемся на данных.

Для учета этого эффекта используется activation-aware постановка. Пусть $x_1, \ldots, x_N$ - входные активации некоторого слоя, собранные на калибровочной выборке. Тогда вместо ошибки весов можно минимизировать ошибку выходов слоя:

$$\sum_{i=1}^{N} \|(W - \widehat{W})x_i\|_2^2.$$

Если обозначить

$$H = \sum_{i=1}^{N} x_i x_i^\top,$$

то этот критерий можно записать как

$$\mathrm{Tr}\left((W - \widehat{W})H(W - \widehat{W})^\top\right).$$

Матрица H описывает статистику входных активаций слоя. Поэтому ошибки в направлениях, часто встречающихся на калибровочных данных, получают больший вес. Для сверточных слоев аналогичная идея применяется к развернутым локальным патчам входных активаций.

В экспериментах калибровочные активации собирались на промптах из выборки PickScore. Для оценки статистики активаций использовались 256 тренировочных промптов, выбранных с фиксированным seed 0. Генерация выполнялась с DDIM-сэмплером, 50 шагами денойзинга и guidance scale, равным 5.0. Для итогового сравнения качества использовались

32 валидационных промпта и три seed: 1234, 4321 и 999, что дает 96 сгенерированных изображений на один вариант квантизации.

Для сравнения качества квантизованных моделей с исходной моделью использовались несколько метрик. Основной принцип оценки состоял в попарном сравнении изображений, полученных исходной и квантизованной моделью при одинаковых промптах и seed.

- **CLIP image-image** - косинусная близость CLIP-эмбеддингов изображения исходной модели и изображения квантизованной модели. Чем выше значение, тем ближе результат квантизованной модели к исходному изображению в семантическом пространстве CLIP.
- **CLIP text-image** - косинусная близость CLIP-эмбеддингов текста и изображения. Эта метрика показывает, насколько сгенерированное изображение соответствует текстовому промпту. Дополнительно рассматривалась разность между CLIP text-image для квантизованной и исходной модели.
- **LPIPS** - перцептуальная дистанция между двумя изображениями. В отличие от пиксельных ошибок, она использует признаки нейронной сети и лучше отражает визуальное сходство. Для LPIPS меньшее значение соответствует более близким изображениям.
- **MSE и MAE** - среднеквадратичная и средняя абсолютная пиксельная ошибка. Эти метрики напрямую измеряют отличие изображений в пространстве пикселей.
- **PSNR** - пиковое отношение сигнала к шуму. Более высокое значение соответствует меньшему уровню пиксельного шума относительно исходного изображения.
- **SSIM** - индекс структурного сходства, учитывающий яркость, контраст и локальную структуру изображений. Более высокое значение означает большее структурное сходство.
- **Время квантизации** - время, необходимое для построения квантизованного представления модели. Эта величина не является метрикой качества генерации, но важна для практического сравнения методов, поскольку некоторые алгоритмы требуют длительной калибровки, обучения кодбуков или дополнительного поиска индексов.

Отдельный учет времени квантизации особенно важен при сравнении скалярных и векторных методов. Скалярная квантизация в общем случае строится быстрее, так как для нее часто достаточно выбрать параметры сетки и независимо округлить веса к ближайшим допустимым значениям. Векторная квантизация обычно дороже: необходимо обучить один или несколько кодбуков, сопоставить фрагменты весов с кодовыми словами, а в некоторых методах дополнительно выполнить уточнение индексов. Поэтому при близком качестве генерации время построения сжатой модели становится самостоятельным критерием выбора метода.

Для ранжирования основных экспериментов использовался агрегированный показатель

$$\text{score} = \text{CLIP}_{I-I} - \text{LPIPS}.$$

Он одновременно поощряет семантическую близость к изображению исходной модели и штрафует за перцептуальное отличие. Дополнительно контролировались значения CLIP text-image, пиксельные метрики и отклонение выходов UNet на фиксированных входах.

3. Рассматриваемые решения

В качестве базовых решений для сравнения были рассмотрены методы двух типов: скалярные и векторные. Скалярные методы заменяют отдельные веса значениями из небольшой числовой сетки. Векторные методы работают с группами весов и поэтому могут использовать более выразительное представление, но обычно требуют более дорогого подбора кодбуков и индексов. Внутри каждой группы методы ниже перечислены от более простых к более сильным и специализированным.

В работах, на которые опирается сравнение, обычно рассматриваются следующие низкобитные режимы:

Метод	Характерный режим квантизации
Q-Diffusion	$4/32$ и $4/8$, где первая величина соответствует битности весов, а вторая - битности активаций
PTQ4DM	post-training квантизация с diffusion-aware calibration; в сравнительных таблицах часто используется режим около 4 бит для весов
BitsFusion	средняя битность весов около 1.99 бит за счет mixed-precision распределения бюджета по слоям
AQUATIC-Diff	additive quantization в режимах $4/8$, $3/8$, $2/8$ и ниже для весов/активаций
VQDM	средняя битность весов 4.15, 3.15 и 2.15 для разных профилей сжатия

Эти значения используются здесь только как ориентир по постановкам из статей; прямое сравнение качества проводится отдельно на едином экспериментальном протоколе.

3.1. Равномерная скалярная PTQ

Наиболее простой базовый подход состоит в равномерной пост-тренировочной квантизации весов. В этом случае модель сначала обучается обычным способом, а затем ее параметры заменяются низкобитным представлением без дополнительного обучения исходной сети. Для каждого тензора, канала или группы весов выбирается диапазон значений, который должен быть представлен конечной сеткой. После этого вычисляется масштаб, а каждое исходное значение округляется к ближайшему допустимому уровню.

Если используется симметричная квантизация, то сетка строится вокруг нуля, а вес w приближенно заменяется величиной

$$\widehat{w} = s \cdot \text{clip}\left(\text{round}\left(\frac{w}{s}\right), -q_{\text{max}}, q_{\text{max}}\right),$$

где s - масштаб квантизации, а q_{max} определяется числом бит. В асимметричном варианте дополнительно используется zero-point, позволяющий сдвигать сетку относительно нуля.

Параметры такой квантизации обычно можно интерпретировать как решение простой задачи восстановления весов:

$$s^* = \arg\min_s \|W - \widehat{W}(s)\|_F^2.$$

В более гибких вариантах масштаб выбирается отдельно для каждого канала или группы, но сама операция остается скалярной: каждый элемент округляется независимо от соседних.

Достоинство такого метода состоит в простоте: он почти не требует калибровочных данных и обычно строится существенно быстрее более сложных схем. Недостаток заключается в том, что веса рассматриваются независимо или почти независимо. Метод не знает, какие

направления во входном пространстве слоя важны, какие слои более чувствительны к ошибке и на каких шагах денойзинга эта ошибка будет особенно заметна. Поэтому равномерная scalar PTQ используется как нижний базовый метод: она показывает качество, достижимое без diffusion-specific предположений.

3.2. PTQ4DM

Первым специализированным скалярным методом является подход из работы *Post-training Quantization on Diffusion Models*. Он исходит из того, что обычная PTQ-калибровка плохо переносится на диффузионные модели. В классификационных сетях калибровочная выборка обычно проходит через модель один раз, и распределение активаций можно считать относительно стабильным. В diffusion-моделях одна и та же denoising-сеть применяется много раз на разных шагах обратного процесса, и распределение ее входов зависит от текущего timestep.

Из-за этого качество квантизации определяется не только весами слоя, но и тем, какие временные шаги использовались при калибровке. Если брать timesteps неудачно, можно хорошо восстановить поведение модели на одних участках траектории и получить большую ошибку на других. PTQ4DM решает эту проблему через timestep-aware calibration. В частности, в работе используется Normally Distributed Time-step Calibration: шаги диффузионного процесса выбираются не равномерно, а по распределению, которое должно лучше покрывать важные состояния модели.

Схематично цель такой калибровки можно записать как

$$\min_{\Theta_q} \sum_{t \in \mathcal{T}_{\text{cal}}} \left\| \epsilon_{\Theta}(x_t, t, c) - \epsilon_{\Theta_q}(x_t, t, c) \right\|_2^2,$$

где ϵ_Θ - исходная noise prediction network, ϵ_{Θ_q} - ее квантизованная версия, а $\mathcal{T}_{\text{cal}}$ - набор калибровочных timesteps.

После сбора калибровочных примеров метод подбирает параметры скалярной квантизации так, чтобы уменьшить расхождение между исходной и квантизованной моделью. Важный момент состоит в том, что оптимизируется не только формальная ошибка округления весов, а поведение сети на данных, возникающих в процессе генерации. Поэтому PTQ4DM сильнее простой scalar PTQ: он остается пост-тренировочным методом, но уже учитывает специфику diffusion inference.

3.3. Q-Diffusion

Следующий скалярный базовый метод описан в работе *Q-Diffusion: Quantizing Diffusion Models*. Как и PTQ4DM, этот метод относится к post-training quantization, но делает более явный акцент на том, что ошибка квантизации в диффузионных моделях накапливается вдоль последовательности denoising-шагов. Даже небольшое искажение выхода noise prediction network на ранних или чувствительных шагах может изменить последующую траекторию генерации.

Q-Diffusion использует timestep-aware calibration: калибровочные данные собираются с учетом разных временных шагов, а параметры квантизаторов подбираются так, чтобы воспроизводить поведение полной модели на этих состояниях. Таким образом, метод стремится аппроксимировать не абстрактный слой сам по себе, а слой в условиях, близких к реальному процессу сэмплирования.

Отдельная часть метода связана с shortcut-соединениями. В diffusion U-Net встречаются активации с неоднородными распределениями, где одна общая равномерная сетка может плохо описывать все значения сразу. Для таких случаев Q-Diffusion предлагает split shortcut

quantization: распределение разбивается на части, и для них используются разные параметры квантизации. Это уменьшает потери информации в shortcut-ветках, которые важны для передачи низкоуровневых признаков через U-Net.

Идею split shortcut quantization можно записать в виде раздельного квантизатора

$$Q_{\text{split}}(y) = \mathbf{1}_{y \le \tau} Q_1(y) + \mathbf{1}_{y > \tau} Q_2(y),$$

где τ - порог разбиения распределения, а Q_1 и Q_2 имеют разные параметры сетки.

В результате Q-Diffusion можно рассматривать как более аккуратную scalar PTQ-схему для diffusion-моделей: она сохраняет общую простоту скалярной квантизации, но добавляет учет timestep-структуры и специальных слоев архитектуры.

3.4. BitsFusion

Наиболее сильный скалярный ориентир среди рассмотренных методов связан с работой *BitsFusion: 1.99 bits Weight Quantization of Diffusion Model*. Главная идея BitsFusion состоит в том, что одинаковая битность для всех слоев является неэффективным использованием бюджета. Разные части U-Net имеют разную чувствительность к квантизации: в одних слоях можно сильно уменьшить число бит без заметной деградации, а в других даже небольшая ошибка приводит к ухудшению генерации.

Поэтому BitsFusion использует mixed-precision quantization. Вместо одного фиксированного числа бит метод распределяет битовый бюджет по слоям. Более чувствительные слои получают больше бит, менее чувствительные - меньше. За счет этого средняя битность модели может быть очень низкой, но критические части сети сохраняются точнее.

Такую постановку удобно описывать как задачу выбора битности слоев при ограничении на средний бюджет:

$$\frac{\sum_\ell n_\ell b_\ell}{\sum_\ell n_\ell} \le \bar{b},$$

где n_ℓ - число параметров в слое ℓ, b_ℓ - выбранная битность этого слоя, а $\bar{b}$ - целевая средняя битность модели.

В полной версии BitsFusion не ограничивается простой калибровкой. Метод использует специальную инициализацию квантизованной модели, feature distillation и обучение квантизационных параметров по сигналу от исходной модели. По смыслу это уже не чистая post-training quantization, а более дорогой training-aware или distillation-based подход. Исходная модель выступает как учитель, а квантизованная модель подстраивается так, чтобы воспроизводить ее внутренние признаки и выходы.

Именно поэтому BitsFusion является сильным, но не самым дешевым базовым методом. Он показывает, насколько далеко можно продвинуть скалярную квантизацию, если разрешить дополнительную оптимизацию после выбора низкобитного представления. Для данной работы он важен как верхний ориентир среди scalar/group low-bit методов, но его вычислительная постановка отличается от полностью пост-тренировочной квантизации без дополнительного обучения.

3.5. Vanilla VQ

Простейший векторный базовый метод основан на классической векторной квантизации. В отличие от скалярных методов, здесь квантуются не отдельные элементы матрицы весов, а группы весов. Матрица разбивается на векторы фиксированной размерности d, после чего на этих векторах обучается кодбук

$$\mathcal{C} = \{c_1, \dots, c_K\}.$$

Обычно такой кодбук можно получить с помощью алгоритма k-means: центроиды подбираются так, чтобы минимизировать среднюю ошибку восстановления обучающих векторов.

Формально это соответствует задаче

$$\min_{\mathcal{C},\{k(i)\}} \sum_i \|v_i - c_{k(i)}\|_2^2.$$

После обучения кодбука каждый фрагмент весов заменяется индексом ближайшего центроида. При хранении модели нужно сохранить сам кодбук и массив индексов. Если векторов много, а кодбук относительно небольшой, это может дать существенное сжатие. При этом VQ способна использовать локальные зависимости между весами внутри одного фрагмента, чего не делает обычная скалярная квантизация.

Ограничение vanilla VQ состоит в том, что каждый фрагмент описывается только одним кодовым словом. Если кодбук мал, одного центроида может быть недостаточно для точного восстановления сложного распределения весов. Если кодбук сильно увеличить, растет стоимость обучения и хранения. Поэтому vanilla VQ полезна не как самый сильный метод, а как минимальный векторный базовый метод: она отделяет выигрыш от самого перехода к кодбукам от выигрыша более сложных многокодбуковых схем.

3.6. AQUATIC-Diff

Более развитая векторная схема рассматривается в работе *AQUATIC-Diff: Additive Quantization for Truly Tiny Compressed Diffusion Models*. Метод использует additive quantization. В обычной VQ каждый фрагмент весов заменяется одним кодовым словом, а в additive quantization он представляется суммой нескольких кодовых слов, выбранных из разных кодбуков:

$$v \approx c_{k_1}^{(1)} + c_{k_2}^{(2)} + \ldots + c_{k_M}^{(M)}.$$

Такое представление значительно увеличивает число возможных восстановленных векторов без необходимости хранить один гигантский кодбук.

Соответствующая задача восстановления имеет вид

$$\min_{\mathcal{C}^{(1)},\ldots,\mathcal{C}^{(M)},\{k_m(i)\}} \sum_i \left\| v_i - \sum_{m=1}^{M} c_{k_m(i)}^{(m)} \right\|_2^2.$$

Для diffusion-моделей это важно по двум причинам. Во-первых, веса разных слоев имеют разную структуру: attention-блоки, линейные проекции и свертки по-разному реагируют на ошибку восстановления. Во-вторых, при очень низкой битности обычной VQ может не хватать выразительности, тогда как сумма нескольких кодовых слов дает более гибкую аппроксимацию. AQUATIC-Diff использует эту идею для получения компактных codebook-based представлений diffusion-моделей.

С вычислительной точки зрения метод дороже vanilla VQ. Нужно обучать несколько кодбуков и выбирать комбинации кодовых слов. Однако по сравнению с одним кодбуком такая схема лучше масштабируется к низким битностям: увеличение числа кодбуков повышает выразительность без экспоненциального роста размера каждого отдельного кодбука. Поэтому AQUATIC-Diff является важным базовым методом для группы подходов, где сжатие достигается не скалярной сеткой, а кодовым представлением весов.

От VQDM, рассматриваемого далее, AQUATIC-Diff отличается постановкой. Его удобно воспринимать как более общий additive-quantization метод для diffusion-моделей: он показывает, что многокодбуковое представление весов может работать при очень низкой битности. Однако метод не является настолько явно привязанным к text-to-image U-Net и задаче сжатия

Stable Diffusion-подобных моделей, как VQDM. Поэтому в данной работе AQUATIC-Diff используется прежде всего как родственный codebook-based подход, а не как полный аналог VQDM.

3.7. VQDM

Самым близким и сильным векторным базовым методом для данной работы является метод из статьи *Accurate Compression of Text-to-Image Diffusion Models via Vector Quantization*. В этом подходе, который далее будем обозначать как VQDM, веса text-to-image diffusion-модели квантуются с помощью многокодбуковой векторной квантизации. Как и в additive quantization, блок весов восстанавливается суммой кодовых слов, выбранных из нескольких кодбуков.

Ключевая сложность такого подхода состоит в выборе индексов. Если для каждого блока нужно выбрать по одному кодовому слову из нескольких кодбуков, число возможных комбинаций быстро растет. Перебор всех вариантов становится слишком дорогим, поэтому VQDM использует приближенный поиск, в частности beam search. Beam search хранит несколько лучших частичных комбинаций и постепенно расширяет их, выбирая более точное восстановление блока весов без полного перебора.

Для одного фрагмента весов задача выбора индексов имеет вид

$$(k_1^*, \ldots, k_M^*) = \arg\min_{k_1,\ldots,k_M} \left\| v - \sum_{m=1}^{M} c_{k_m}^{(m)} \right\|_2^2 .$$

Именно эта комбинаторная часть делает многокодбуковую векторную квантизацию существенно дороже обычной scalar PTQ.

После послойного построения кодбуков и индексов в полной версии VQDM может использоваться дополнительное дообучение. Его смысл состоит в том, чтобы уменьшить расхождение между исходной и квантизованной diffusion-моделью уже на уровне предсказаний модели, а не только на уровне отдельных матриц весов. Это делает метод сильнее, но одновременно повышает стоимость запуска: кроме обучения кодбуков и beam search требуется еще этап оптимизации квантизованной модели.

Таким образом, VQDM и AQUATIC-Diff используют близкую математическую идею - восстановление блока весов суммой кодовых слов из нескольких кодбуков. Различие состоит в уровне специализации и стоимости. AQUATIC-Diff важен как общий additive-quantization подход для diffusion-моделей, тогда как VQDM специально строится для text-to-image diffusion-моделей и делает более дорогой подбор кодов через beam search. Именно поэтому VQDM является более близким базовым методом для данной работы, но одновременно сильнее мотивирует поиск более дешевой векторной схемы.

4. Условия экспериментов

В качестве основной модели в экспериментах используется Stable Diffusion 1.5. Полный конвейер такой модели состоит из нескольких частей: текстового кодировщика, U-Net, выполняющего итеративное предсказание шума, и VAE, переводящего латентное представление в изображение. В данной работе квантизация применяется только к U-Net части модели, поскольку именно она является основной вычислительной частью процесса генерации и многократно вызывается на каждом шаге денойзинга. Текстовый кодировщик и VAE оставляются в исходном представлении, чтобы сравнение отражало влияние квантизации denoising-сети, а не сразу нескольких независимых компонент конвейера.

Практическое сравнение проводится на среднем блоке U-Net, то есть на модуле mid_block. Этот блок находится в наиболее сжатом латентном разрешении U-Net и содержит две ResNet-части и attention/transformer-часть между ними. Такая область удобна для экспериментов: она достаточно мала, чтобы быстро запускать разные методы, но при этом содержит разные типы слоев, характерные для полной U-Net архитектуры.

В полном варианте рассматриваются 18 квантуемых слоев mid_block:

- 10 линейных слоев transformer-блока: проекции self-attention attn1.to_q, attn1.to_k, attn1.to_v, attn1.to_out.0, аналогичные проекции cross-attention attn2.to_q, attn2.to_k, attn2.to_v, attn2.to_out.0, а также два слоя feed-forward блока ff.net.0.proj и ff.net.2;
- 6 сверточных слоев: входная и выходная проекции attention-блока proj_in, proj_out, а также пары conv1, conv2 в двух ResNet-блоках;
- 2 линейные проекции timestep embedding: по одному слою time_emb_proj в каждом из двух ResNet-блоков.

Для векторных методов дополнительно рассматривается вариант с 16 слоями. Он отличается от полного режима тем, что не квантует две проекции time_emb_proj. Такой выбор нужен не потому, что эти слои не важны, а для сопоставления с VQDM-style постановкой: исходный набор целевых слоев VQDM ориентирован на основные attention, feed-forward и convolution операторы U-Net и не включает timestep embedding projections. Поэтому для векторных методов в работе фиксируются оба варианта: полный режим на 18 слоях и режим на 16 слоях, близкий к исходной экспериментальной постановке VQDM.

Во всех экспериментах рассматривается post-training постановка без последующего fine-tuning исходной diffusion-модели. Это важно для интерпретации результатов. Часть методов в оригинальных статьях допускает дополнительное дообучение или distillation после построения квантизованного представления. Например, полные версии VQDM и BitsFusion могут улучшать качество за счет оптимизации квантизованной модели относительно исходной. В данной работе этот этап не используется: сравниваются методы, применяемые после обучения модели и использующие только калибровку, обучение кодбуков или подбор параметров квантизации. Такое ограничение делает запуск дешевле и ближе к practical PTQ-сценарию, но для методов, рассчитанных на последующее дообучение, может ухудшать качество относительно результатов из оригинальных работ.

Для скалярных методов используются целевые битности 2, 3 и 4 бита на вес. Эти режимы соответствуют стандартной low-bit постановке scalar PTQ и позволяют проследить, как качество меняется при увеличении битового бюджета. Скалярные методы запускаются на варианте с 18 слоями, то есть на той же полной группе слоев, которая используется для основного сравнения.

Для векторных методов используются целевые VQDM-style профили 2.15, 3.15 и 4.15 bpp. Эти названия профилей соответствуют схемам с 2, 3 и 4 кодбуками при $K = 256$ и $d = 8$: один индекс занимает $\log_2 256 = 8$ бит и описывает 8 весов, поэтому один кодбук дает примерно 1 index-bpp. Дробная часть в названии профиля отражает ожидаемые накладные расходы на хранение кодбуков и служебной информации. При этом в таблицах ниже указывается не название профиля, а фактическая оценка bpp, пересчитанная для конкретного набора квантуемых слоев. Поэтому профиль 2.15 может иметь фактическую стоимость около 2.02 bpp, если overhead кодбуков оказывается мал относительно числа весов.

Предлагаемый RVQ-подход настраивается на те же целевые профили, что и VQDM. Поэтому различие между ними состоит не в выделенном битовом бюджете, а в способе построения и выбора кодов: VQDM использует более дорогой совместный поиск комбинаций кодовых слов, тогда как RVQ выбирает коды последовательно по остаткам. Для каждого векторного профиля рассматриваются оба режима по числу слоев: 16 слоев для сопоставления

с VQDM-style постановкой и 18 слоев для честного сравнения со скалярными методами на одинаковой области U-Net.

Таким образом, экспериментальная сетка разделяется на две части. Скалярные базовые методы сравниваются при 2, 3 и 4 битах на 18 слоях. Векторные методы сравниваются при целевых профилях 2.15, 3.15 и 4.15 как на 16, так и на 18 слоях; фактическая стоимость хранения затем дополнительно пересчитывается и приводится в таблицах. Во всех случаях генерация и оценка выполняются по единому протоколу, описанному выше: используются одинаковые промпты, seed, число шагов сэмплирования и набор метрик.

5. Результаты запуска базовых методов

В этом разделе сравниваются базовые методы, описанные выше. В таблицах **CLIP I-I** означает CLIP image-image между изображениями исходной и квантизованной модели, а **CLIP T-I** - CLIP text-image для изображений квантизованной модели. **Score** соответствует величине $\text{CLIP}_{I-I} - \text{LPIPS}$. Время T_q измеряется в секундах и включает построение квантизованного представления, а также калибровку, если она требуется методу; время генерации и вычисления метрик в T_q не включается.

5.1. Скалярные базовые методы

Скалярные методы запускались на полном наборе из 18 слоев mid_block. Результаты приведены для трех целевых битовых режимов.

Метод	bpp	CLIP I-I	CLIP T-I	LPIPS	Score	T_q, с
Профиль около 2 бит						
Scalar PTQ	2.01	0.605	0.275	0.472	0.133	0.03
Q-Diffusion	2.01	0.668	0.298	0.421	0.247	59.4
PTQ4DM	2.01	0.685	0.306	0.429	0.256	63.8
BitsFusion	2.05	0.768	0.312	0.300	0.468	88.4
Профиль около 3 бит						
Scalar PTQ	3.01	0.755	0.317	0.323	0.432	0.02
Q-Diffusion	3.01	0.807	0.318	0.239	0.568	59.3
PTQ4DM	3.01	0.799	0.318	0.237	0.562	63.8
BitsFusion	3.04	0.829	0.319	0.218	0.611	88.4
Профиль около 4 бит						
Scalar PTQ	4.01	0.858	0.318	0.187	0.671	0.23
Q-Diffusion	4.01	0.872	0.322	0.150	0.722	59.3
PTQ4DM	4.01	0.876	0.318	0.157	0.719	63.8
BitsFusion	4.01	0.871	0.321	0.158	0.713	88.4

По этим результатам видно, что простая равномерная scalar PTQ заметно уступает diffusion-aware методам, особенно при 2 и 3 битах. На низких битностях лучшим скалярным базовым методом оказывается BitsFusion, а в режиме около 4 бит наибольший Score показывает Q-Diffusion, очень близко к нему находится PTQ4DM. При этом scalar PTQ почти не требует времени на построение квантизованной модели, тогда как Q-Diffusion, PTQ4DM и BitsFusion тратят дополнительное время на калибровку и подбор параметров.

5.2. Векторные базовые методы на 16 слоях

Для векторных методов сначала рассматривается VQDM-style режим на 16 слоях, в котором не квантуются две проекции time_emb_proj. Такой вариант нужен для сопоставления с исходной постановкой VQDM.

Метод	bpp	CLIP I-I	CLIP T-I	LPIPS	Score	T_q, с
Профиль около 2 бит						
VQDM	2.02	0.847	0.312	0.229	0.618	859.8
Vanilla VQ	2.00	0.766	0.308	0.310	0.456	0.96
AQUATIC-Diff	2.01	0.773	0.309	0.292	0.481	78.0
Профиль около 3 бит						
VQDM	3.02	0.901	0.309	0.150	0.751	1157.2
Vanilla VQ	3.00	0.829	0.321	0.212	0.617	1.82
AQUATIC-Diff	3.02	0.811	0.323	0.238	0.573	78.9
Профиль около 4 бит						
VQDM	4.03	0.919	0.308	0.119	0.800	1956.6
Vanilla VQ	4.00	0.874	0.321	0.156	0.717	2.84
AQUATIC-Diff	4.02	0.863	0.321	0.170	0.692	83.0

На 16 слоях VQDM оказывается лучшим векторным базовым методом во всех трех битовых режимах. Разница особенно заметна при 2 битах: многокодбуковый beam-search подбор кодов дает существенно более высокое качество, чем vanilla VQ и AQUATIC-Diff. Цена этого выигрыша - значительно большее время построения квантизованного представления.

5.3. Векторные базовые методы на 18 слоях

Далее те же векторные методы сравниваются на полном наборе из 18 слоев mid_block. Этот режим напрямую сопоставим со скалярными базовыми методами.

Метод	bpp	CLIP I-I	CLIP T-I	LPIPS	Score	T_q, с
Профиль около 2 бит						
VQDM	2.02	0.831	0.305	0.244	0.586	1857.1
Vanilla VQ	2.00	0.763	0.311	0.314	0.449	1.02
AQUATIC-Diff	2.01	0.760	0.311	0.306	0.453	65.4
Профиль около 3 бит						
VQDM	3.02	0.885	0.308	0.164	0.721	2846.8
Vanilla VQ	3.01	0.828	0.319	0.212	0.617	1.91
AQUATIC-Diff	3.02	0.819	0.322	0.229	0.591	66.4
Профиль около 4 бит						
VQDM	4.03	0.926	0.310	0.101	0.825	3314.0
Vanilla VQ	4.00	0.881	0.317	0.144	0.737	1.57
AQUATIC-Diff	4.02	0.860	0.317	0.175	0.685	67.6

На 18 слоях сохраняется та же картина: VQDM дает лучшее качество среди векторных базовых методов, но требует на порядки больше времени. Vanilla VQ оказывается неожиданно быстрым, однако его качество заметно ниже VQDM. AQUATIC-Diff находится между ними по вычислительной стоимости, но в данном экспериментальном протоколе не превосходит vanilla VQ по Score.

Важно подчеркнуть, что этот результат не означает слабость полной версии AQUATIC-Diff из оригинальной работы. В данной таблице используется AQUATIC-Diff-like базовый метод, то есть упрощенная адаптация основной additive-quantization идеи к нашему mid-block протоколу. Полная постановка статьи богаче: она ориентирована на квантизацию более широкой части diffusion-модели и использует дополнительные процедуры настройки/калибровки. В наших экспериментах, чтобы сохранить единый no-fine-tuning протокол, эти дополнительные этапы не используются. Поэтому AQUATIC-Diff в таблице следует интерпретировать как проверку самой идеи additive codebook quantization в наших условиях, а не как полную репродукцию метода из статьи.

5.4. Лучшие базовые методы на 18 слоях

Для итогового сравнения ниже приведены четыре лучших базовых метода на полном наборе из 18 слоев для каждого битового профиля. Отбор выполнялся по Score.

Профиль	Метод	bpp	CLIP I-I	CLIP T-I	LPIPS	Score	T_q, с
Профиль около 2 бит							
2	VQDM	2.02	0.831	0.305	0.244	0.586	1857.1
2	BitsFusion	2.05	0.768	0.312	0.300	0.468	88.4
2	AQUATIC-Diff	2.01	0.760	0.311	0.306	0.453	65.4
2	Vanilla VQ	2.00	0.763	0.311	0.314	0.449	1.02
Профиль около 3 бит							
3	VQDM	3.02	0.885	0.308	0.164	0.721	2846.8
3	Vanilla VQ	3.01	0.828	0.319	0.212	0.617	1.91
3	BitsFusion	3.04	0.829	0.319	0.218	0.611	88.4
3	AQUATIC-Diff	3.02	0.819	0.322	0.229	0.591	66.4
Профиль около 4 бит							
4	VQDM	4.03	0.926	0.310	0.101	0.825	3314.0
4	Vanilla VQ	4.00	0.881	0.317	0.144	0.737	1.57
4	Q-Diffusion	4.01	0.872	0.322	0.150	0.722	59.3
4	PTQ4DM	4.01	0.876	0.318	0.157	0.719	63.8

Итоговый вывод состоит в том, что среди базовых методов наилучшее качество дает VQDM: при близкой битности он заметно превосходит лучшие скалярные методы по CLIP I-I, LPIPS и Score. Однако этот выигрыш достигается ценой очень дорогого построения кодов, связанного с многокодбуковой векторной квантизацией и beam search. Простая Vanilla VQ, напротив, строится быстро и также оказывается конкурентоспособной на 3–4 битах, но не достигает качества VQDM. Поэтому дальнейшее улучшение векторной квантизации должно сохранять часть выразительности VQDM, но избегать его полной вычислительной стоимости.

6. Предлагаемый RVQ-подход

После анализа базовых методов основной вопрос состоит в том, можно ли сохранить часть качества VQDM, но заметно уменьшить стоимость построения кодов. VQDM дает сильный результат, однако его многокодбуковая схема требует дорогого подбора комбинаций кодовых слов. В данной работе рассматривается более простая residual vector quantization (RVQ), в которой кодбуки строятся и применяются последовательно. RVQ настраивается на те же целевые профили 2.15, 3.15 и 4.15 bpp, что и VQDM, поэтому сравнение проверяет не выигрыш за счет большего битового бюджета, а замену дорогого совместного поиска более дешевой остаточной процедурой.

6.1. Постановка задачи RVQ

Пусть фрагмент весов представлен вектором $v \in \mathbb{R}^d$. В RVQ он аппроксимируется суммой M кодовых слов из последовательности кодбуков:

$$\hat{v} = \sum_{m=1}^{M} c_{k_m}^{(m)}, \qquad c_{k_m}^{(m)} \in \mathcal{C}^{(m)}.$$

Главное отличие от более дорогой additive vector quantization состоит в способе выбора кодов. В AVQ-подходе для каждого вектора нужно искать хорошую комбинацию индексов

$(k_1, \ldots, k_M)$ совместно. Именно эта часть приводит к beam search: на каждом шаге поддерживается несколько лучших частичных комбинаций, которые затем расширяются следующими кодбуками. Такой поиск повышает качество, но быстро увеличивает время работы.

В RVQ кодбуки используются жадно. Сначала первый кодбук аппроксимирует исходный вектор, затем второй кодбук аппроксимирует остаток, затем третий кодбук аппроксимирует новый остаток и так далее:

$$r^{(0)} = v, \qquad k_m = \arg\min_k \left\| r^{(m-1)} - c_k^{(m)} \right\|_2^2, \qquad r^{(m)} = r^{(m-1)} - c_{k_m}^{(m)}.$$

После M стадий получается $\hat{v} = v - r^{(M)}$. Такая схема значительно дешевле, потому что на каждой стадии нужно решить обычную задачу ближайшего кодового слова, а не искать комбинацию из нескольких кодбуков. Поэтому RVQ проще масштабировать на большее число вариантов K, M и профилей битности.

Цена этой дешевизны состоит в жадности. Ранние кодовые слова выбираются без знания того, какие ошибки смогут исправить следующие стадии. Если первая стадия выбрала неоптимальное направление, последующие кодбуки работают уже с получившимся остатком и не могут полностью пересобрать всю комбинацию. Поэтому при одинаковом числе кодбуков RVQ в общем случае слабее AVQ/VQDM.

В рассматриваемых целевых профилях при фиксированном размере кодбука $K = 256$ и размерности фрагмента $d = 8$ один кодбук дает примерно один бит на параметр: один индекс занимает $\log_2 K = 8$ бит и описывает 8 весов. Поэтому профили 4.15, 3.15 и 2.15 bpp соответствуют примерно $M = 4$, $M = 3$ и $M = 2$ кодбукам с учетом небольшого overhead на хранение центроидов. Фактическая bpp после учета overhead пересчитывается отдельно для конкретного набора слоев, как и для VQDM. В этом месте снижение битности особенно болезненно: уменьшается не только точность индекса, но и само число слагаемых в RVQ-разложении. При переходе от четырех кодбуков к двум модель теряет половину аддитивных компонент, поэтому качество на низких bpp падает существенно сильнее.

6.2. Activation-aware функционал

Обычная постановка VQ минимизирует ошибку восстановления весов. Для diffusion-модели это не всегда соответствует ошибке работы слоя, потому что разные координаты входа используются с разной частотой и имеют разные корреляции. Поэтому в предлагаемом подходе используется activation-aware функционал.

Рассмотрим линейный слой с матрицей весов $W \in \mathbb{R}^{o \times p}$. Пусть на калибровочных данных собрана матрица входных активаций $X \in \mathbb{R}^{n \times p}$. Если квантованная матрица весов равна $\hat{W}$, то ошибка выходов слоя равна

$$XW^\top - X\hat{W}^\top = X(W - \hat{W})^\top.$$

Соответствующий квадратичный функционал можно записать через матрицу

$$H = \frac{1}{n} X^\top X :$$

$$\mathcal{L}_{\text{act}}(\hat{W}) = \frac{1}{n} \left\| X(W - \hat{W})^\top \right\|_F^2 = \text{Tr}\left((W - \hat{W}) H (W - \hat{W})^\top \right).$$

Для сверточных слоев используется та же идея, но входные активации предварительно приводятся к матричному виду через разворачивание локальных окон свертки. Поэтому H можно понимать как эмпирическую матрицу вторых моментов входа слоя.

На практике полная матрица H может быть большой, поэтому обучение кодбуков удобно вести по блокам координат. Если S_j обозначает очередной блок из d координат, то локальная часть функционала имеет вид

$$\mathcal{L}_{\text{block}} = \sum_{a,j} \left(w_{a,S_j} - \hat{w}_{a,S_j}\right)^\top H_{S_j,S_j} \left(w_{a,S_j} - \hat{w}_{a,S_j}\right),$$

где a индексирует выходные каналы или строки матрицы весов. На этом этапе базовый RVQ строит кодбуки и выбирает индексы только в рамках последовательной остаточной процедуры. Дополнительное уточнение дискретных кодов после построения RVQ не входит в исходную постановку метода; оно рассматривается далее как отдельная модификация.

6.3. Ортогональное преобразование

Следующая гипотеза состоит в том, что перед RVQ полезно применить ортогональное преобразование координат. Интуиция здесь следующая. RVQ работает с короткими блоками весов и жадно аппроксимирует остатки. Если энергия ошибки сконцентрирована в отдельных координатах или сильно зависит от корреляций между координатами, то блочная жадная схема может тратить кодовые слова неэффективно. Ортогональное преобразование не меняет норму вектора, но перераспределяет координаты и меняет геометрию задачи, которую видит кодбук.

Пусть R – ортогональная матрица, то есть $R^\top R = I$. Для вектора весов v будем квантовать не сам v, а преобразованный вектор

$$z = Rv.$$

После RVQ получаем $\hat{z}$, а затем возвращаемся в исходное пространство:

$$\hat{v} = R^\top \hat{z}.$$

Если используется activation-aware функционал с матрицей H, то ошибка в преобразованном пространстве имеет вид

$$(v - \hat{v})^\top H (v - \hat{v}) = (z - \hat{z})^\top \left(RHR^\top\right)(z - \hat{z}).$$

Таким образом, ортогональное преобразование не просто меняет сами векторы весов, но и переводит матрицу вторых моментов в новую систему координат:

$$H_R = RHR^\top.$$

Если бы квантование было непрерывной оптимизацией без ограничений на кодбуки, такая замена координат не давала бы выигрыша. Однако RVQ ограничена конечными кодбуками, блочной структурой и жадным выбором кодов, поэтому выбор координат может заметно влиять на качество.

В качестве простого ортогонального преобразования рассматривается Hadamard-преобразование. Матрица Адамара строится рекурсивно:

$$\tilde{H}_1 = [1], \qquad \tilde{H}_{2B} = \begin{pmatrix} \tilde{H}_B & \tilde{H}_B \\ \tilde{H}_B & -\tilde{H}_B \end{pmatrix}, \qquad H_B = \frac{1}{\sqrt{B}} \tilde{H}_B.$$

Нормированная матрица H_B ортогональна и содержит только элементы $\pm 1/\sqrt{B}$. В экспериментах она применяется блочно по координатам входного пространства слоя:

$$R_B = \text{diag}(H_B, H_B, \ldots, H_B).$$

Кроме чистого Hadamard-преобразования рассматривался вариант с дополнительной диагональной матрицей знаков:

$$R_{B,s} = \text{diag}(H_B, \ldots, H_B)D_s, \qquad D_s = \text{diag}(s_1, \ldots, s_p), \qquad s_i \in \{-1, 1\}.$$

Такая матрица также ортогональна, но меняет ориентацию координат перед смешиванием Hadamard-блоком. В эксперименте рассматривались блоки $B = 8$, 16, 32, 64, 128 и 256 как без дополнительной диагонали знаков, так и с ней.

6.4. Предварительные эксперименты с Hadamard

Первая экспериментальная проверка ортогонального преобразования была сделана в максимально изолированной постановке. Квантизовались только 10 линейных слоев mid-block, без сверток; размер фрагмента был фиксирован как $d = 8$, размер каждого кодбука - $K = 256$. Для целевых профилей 2.15, 3.15 и 4.15 bpp это соответствует 2, 3 и 4 RVQ-стадиям соответственно. Такой запуск дешевле полного mid-block, но сохраняет основную задачу: проверить, меняет ли ортогональное перемешивание координат качество базовой RVQ-аппроксимации.

В эксперименте сравнивался вариант без преобразования, обозначаемый noH, и Hadamard-преобразования с размерами блоков 8, 16, 32, 64, 128 и 256. Для каждого размера блока запускались два варианта: без случайной диагонали знаков и с ней. Всего было проверено 39 конфигураций: 3 битовых профиля и 13 вариантов преобразования в каждом.

Профиль	Лучшее преобразование	Score	LPIPS	CLIP I-I	CLIP T-I	T_q, с
2.15	noH	0.5890	0.2406	0.8297	0.3066	304.0
3.15	H16, без знаков	0.7066	0.1804	0.8871	0.3108	456.2
4.15	noH	0.8061	0.1121	0.9182	0.3102	608.9

По этой таблице видно, что Hadamard не является безусловным улучшением. На низком профиле 2.15 bpp и на высоком профиле 4.15 bpp лучшим оказался исходный вариант без ортогонального преобразования. Единственный профиль, где Hadamard дал заметный выигрыш, - 3.15 bpp: блок 16 без случайных знаков улучшил Score с 0.6926 до 0.7066.

Профиль	Преобразование	Score	Δ Score	Δ LPIPS	Δ CLIP I-I
3.15	H8, со знаками	0.6954	+0.0029	+0.0029	+0.0058
3.15	H16, без знаков	0.7066	+0.0141	-0.0001	+0.0140
3.15	H32, без знаков	0.6993	+0.0068	-0.0000	+0.0068

В таблице выше приведены только те Hadamard-варианты, которые улучшили Score относительно noH внутри своего профиля. Все такие случаи относятся к 3.15 bpp. При 2.15 bpp ближайший Hadamard-вариант, H8 без знаков, получил Score 0.5832 против 0.5890 у noH. При 4.15 bpp ближайший Hadamard-вариант, H8 со знаками, получил Score 0.7978 против 0.8061 у noH.

Профиль	noH	H8	H16	H32	H64	H128	H256
2.15	0.5890	0.5832	0.5723	0.5552	0.5545	0.5417	0.5313
3.15	0.6926	0.6954	0.7066	0.6993	0.6622	0.6745	0.6611
4.15	0.8061	0.7978	0.7809	0.7863	0.7824	0.7865	0.7424

В последней таблице для каждого размера блока показан лучший вариант по знакам. Она показывает, что большие блоки не дают устойчивого преимущества. Блок 256 во всех профилях заметно слабее лучших вариантов; блоки 64 и 128 также не выглядят надежными кандидатами. Наиболее разумная область поиска - малые и средние блоки 8, 16 и 32.

Итак, первая гипотеза подтвердилась только частично. Ортогональное преобразование действительно может улучшить RVQ, но эффект зависит от битового профиля и размера блока. В фиксированной постановке с $K = 256$ лучший результат на 3.15 bpp дает H16 без случайных знаков, тогда как на 2.15 и 4.15 bpp преобразование ухудшает Score. Поэтому Hadamard следует рассматривать не как обязательный шаг алгоритма, а как отдельную настройку, которую нужно выбирать по профилю.

6.5. Влияние размера кодбука

После проверки ортогонального преобразования отдельно рассматривалось влияние размера кодбука K. В RVQ при фиксированном числе кодбуков увеличение K расширяет множество допустимых кодовых слов на каждой стадии и поэтому должно уменьшать ошибку восстановления. Обратная сторона состоит в том, что для хранения индекса каждого кодового слова требуется больше бит:

$$b_{\text{index}} = \frac{M \log_2 K}{d},$$

где M - число RVQ-стадий, а d - размерность фрагмента. Поэтому увеличение K нельзя рассматривать как бесплатное улучшение: оно меняет фактический битовый профиль.

Для быстрой проверки был проведен запуск на 10 линейных слоях mid-block при фиксированном $d = 8$. После обучения кодбуков индексы дополнительно не уточнялись: использовалась базовая последовательная RVQ-процедура. Сравнивались варианты без ортогонального преобразования и с Hadamard-блоком $H8$. Значения K выбирались так, чтобы проверить направление эффекта и при этом не слишком далеко уходить от целевых профилей.

Цель	K	M	Факт. bpp	Score noH	Score H8
2.15	128	2	1.76	0.5166	0.5234
2.15	256	2	2.02	0.5847	0.5584
2.15	512	2	2.29	0.6142	0.6090
3.15	128	4	3.52	0.7145	0.7338
3.15	256	3	3.03	0.7138	0.7111
3.15	512	3	3.44	0.7631	0.7456

Результаты подтверждают ожидаемую тенденцию: более крупный кодбук в среднем повышает качество, особенно если сравнивать режимы с близким числом RVQ-стадий. Однако вместе с этим растет фактическая битность. Поэтому в дальнейшем размер кодбука нужно подбирать как часть битового профиля, а не просто увеличивать его до максимума: иначе сравнение с другими методами становится нечестным.

6.6. Рекодинг после RVQ на полной матрице $X^\top X$

В activation-aware RVQ кодбуки обучаются не по евклидовой ошибке весов, а по квадратичному функционалу, зависящему от матрицы активаций $X^\top X$. Однако во время обучения кодбуков эта матрица используется блочно: весовая матрица разбивается на короткие фрагменты размера d, и для каждого фрагмента берется соответствующий блок $X^\top X$. Такой подход делает обучение дешевле и позволяет масштабировать метод, но игнорирует внедиагональные взаимодействия между разными фрагментами строки веса.

После обучения RVQ можно сделать дополнительный дискретный шаг: зафиксировать все кодбуки и перевыбрать только индексы кодовых слов, оценивая замену по полной матрице $X^\top X$. Если обозначить ошибку строки веса как $e = \hat{w} - w$, то локальная замена одного кода проверяется по изменению полного функционала

$$\Delta\mathcal{L} = (\hat{w} + \Delta w - w)^\top X^\top X(\hat{w} + \Delta w - w) - (\hat{w} - w)^\top X^\top X(\hat{w} - w).$$

На практике это реализуется как coordinate descent по уже выбранным кодам: для каждой RVQ-стадии и каждого фрагмента перебираются возможные коды, выбирается замена с наибольшим уменьшением функционала, затем обновляется текущая ошибка. Важно, что это не является файнтюнингом в смысле обучения модели: веса нейросети и кодбуки не оптимизируются градиентным методом, а меняются только дискретные индексы при фиксированном наборе кодовых слов.

Для быстрой проверки был проведен запуск на 10 линейных слоях mid-block. В этой части фиксировалась размерность фрагмента $d = 8$, но рассматривались несколько размеров кодбука K. Это позволяет проверить рекодинг не только в одной точке, а в нескольких близких битовых режимах. Сравнивались одинаковые конфигурации до и после одного прохода full-XTX рекодинга.

Цель	K	M	bpp	Преобр.	Score без	Score с	Δ Score	Δ CLIP I-I
2.15	128	2	1.76	noH	0.5166	0.5257	+0.0090	+0.0024
2.15	128	2	1.76	H8	0.5234	0.5486	+0.0252	+0.0073
2.15	256	2	2.02	noH	0.5847	0.5839	-0.0008	+0.0063
2.15	256	2	2.02	H8	0.5584	0.5842	+0.0258	+0.0156
2.15	512	2	2.29	noH	0.6142	0.6389	+0.0247	+0.0062
2.15	512	2	2.29	H8	0.6090	0.6193	+0.0102	+0.0019
3.15	128	4	3.52	noH	0.7145	0.7273	+0.0127	+0.0055
3.15	128	4	3.52	H8	0.7338	0.7356	+0.0018	-0.0024
3.15	256	3	3.03	noH	0.7138	0.7024	-0.0115	-0.0023
3.15	256	3	3.03	H8	0.7111	0.6872	-0.0238	-0.0081
3.15	512	3	3.44	noH	0.7631	0.7511	-0.0120	+0.0011
3.15	512	3	3.44	H8	0.7456	0.7810	+0.0354	+0.0110

В этой серии рекодинг улучшил Score в 8 из 12 парных сравнений. Среднее изменение Score составило $+0.0081$, среднее изменение CLIP I-I - $+0.0037$, а среднее изменение LPIPS было отрицательным, то есть также соответствовало улучшению. При этом отдельные отрицательные строки сохраняются: уменьшение внутреннего функционала не гарантирует рост каждой внешней метрики в каждой конкретной конфигурации. В среднем для этих вариантов время построения квантизованного представления выросло примерно на 2.9%.

Дополнительно был проведен быстрый запуск с уже включенным full-XTX рекодингом, где при тех же $d = 8$ и $K = 256$ сравнивались отсутствие ортогонального преобразования и несколько предварительно отобранных Hadamard-блоков. Он нужен не для измерения вклада рекодинга сам по себе, а для проверки того, что рекодинг совместим как с noH, так и с Hadamard-вариантами.

Профиль	noH	H8	H16	Лучший вариант
4.15 bpp	0.8010	0.7873	0.7851	noH
3.15 bpp	0.7101	0.7121	0.7051	H8
2.15 bpp	0.5688	0.5627	0.5907	H16

Таким образом, при фиксированных $d = 8$ и $K = 256$ рекодинг использовался не только в варианте без ортогонального преобразования, но и вместе с Hadamard-блоками 8 и 16. Лучший блок зависел от битового профиля: на 4.15 bpp сильнее оказался noH, на 3.15 bpp - $H8$, а на 2.15 bpp - $H16$. Это дало повод продолжить использование ортогонального преобразования в дальнейших экспериментах.

Отдельно логировалось изменение самого full-XTX функционала во время рекодинга. По 150 вызовам coordinate descent среднее относительное улучшение целевого функционала составило 47.53%, медианное - 50.26%, а диапазон значений был от 21.87% до 58.48%. Это показывает, что post-RVQ рекодинг действительно исправляет значительную часть ошибки, оставшейся из-за блочного обучения RVQ. Поэтому в дальнейших запусках этот шаг рассматривается как штатная часть метода, а не как отдельный дорогостоящий этап обучения.

6.7. Проверка увеличенной размерности фрагмента

Еще одна естественная гипотеза состояла в увеличении размерности фрагмента с $d = 8$ до $d = 16$. Интуитивно более длинный вектор может захватывать больше локальной структуры в строке матрицы весов: одно кодовое слово описывает не восемь, а шестнадцать соседних координат. Если в этих координатах есть устойчивые зависимости, более крупный фрагмент мог бы уменьшить ошибку восстановления.

При фиксированном размере кодбука $K = 256$ такая замена меняет соотношение между размерностью фрагмента и числом стадий. Один индекс по-прежнему занимает 8 бит, но теперь описывает 16 весов. Поэтому при том же index-bpp можно примерно вдвое увеличить число RVQ-стадий M:

$$\frac{M \log_2 K}{d} = \text{const}.$$

Для профиля 4.15 bpp использовались $M = 4$ стадии при $d = 8$ и $M = 8$ стадий при $d = 16$; для 3.15 bpp - $M = 3$ и $M = 6$; для 2.15 bpp - $M = 2$ и $M = 4$. Это должно было быть преимуществом варианта $d = 16$: большее число RVQ-стадий дает больше аддитивных компонент и позволяет последовательнее аппроксимировать остаток.

Проверка проводилась на 10 линейных слоях mid-block при $K = 256$. Сравнивались варианты с Hadamard-блоками 8 и 16, а также одна завершенная точка без ортогонального преобразования. Full-XTX рекодинг был включен одинаково для обеих размерностей, поэтому сравнение отражает поведение всего тогдашнего RVQ-пайплайна, а не изолированно только обучения кодбуков.

Профиль	Преобр.	Блок	Score $d = 8$	Score $d = 16$	Δ Score	$T_{d=16}/T_{d=8}$
4.15	H	8	0.7927	0.7861	+0.0066	1.95
3.15	H	8	0.6892	0.6579	+0.0313	1.97
2.15	H	8	0.5803	0.5707	+0.0096	1.95
4.15	H	16	0.7842	0.7473	+0.0370	1.95
3.15	H	16	0.6883	0.6613	+0.0269	1.97
2.15	H	16	0.5859	0.5587	+0.0271	1.96
4.15	noH	–	0.7761	0.7565	+0.0196	1.96

Положительное значение Δ означает преимущество $d = 8$. На всех завершенных парных сравнениях $d = 8$ оказался лучше по Score; аналогичная картина наблюдалась по LPIPS и CLIP I-I. При этом время построения кодбуков для $d = 16$ выросло почти вдвое. Следовательно, исходная гипотеза не подтвердилась: даже дополнительное число RVQ-стадий при той же index-bpp не компенсировало ухудшение, связанное с более высокой размерностью фрагмента при $K = 256$. Поэтому в дальнейших экспериментах основным вариантом выбран $d = 8$, а $d = 16$ остается отрицательной проверкой гипотезы.

6.8. Распределение размера кодбуков по RVQ-стадиям

Предыдущие эксперименты показали, что увеличение K может заметно улучшать качество, но равномерное увеличение размера всех кодбуков уводит фактический bpp от целевого профиля. Поэтому была проверена более аккуратная идея: менять K не одновременно у всех стадий, а перераспределять кодовый бюджет между RVQ-стадиями. Для профиля с M кодбуками битовая стоимость индексов определяется суммой

$$b_{\text{index}} = \frac{1}{d} \sum_{m=1}^{M} \log_2 K_m.$$

Значит, при фиксированном $d = 8$ можно заменить равномерный профиль, например $[256, 256]$, на $[64, 1024]$ или $[512, 128]$ и сохранить index-bpp. Такая схема позволяет проверить, одинаково ли важны разные RVQ-стадии. Ранние стадии работают с более крупным остатком, поздние исправляют уже оставшуюся ошибку, поэтому оптимальный размер кодбука может зависеть от номера стадии.

Эксперимент проводился на тех же 10 линейных слоях mid-block. В таблице приведены проверенные профили распределения K по стадиям при $d = 8$ для низкого и среднего битового бюджета. При этом финальный full-XTX рекодинг был включен, как в основной RVQ-процедуре, поэтому результаты нужно понимать как проверку распределения K внутри рабочей версии метода.

Цель	$K_1, \ldots, K_M$	Преобр.	bpp	Score	LPIPS	CLIP I-I
2.15	$[1024, 64]$	noH	2.04	0.5810	0.2454	0.8265
2.15	$[1024, 64]$	H8	2.04	0.5774	0.2479	0.8253
2.15	$[512, 128]$	noH	2.03	0.5898	0.2451	0.8349
2.15	$[512, 128]$	H8	2.03	0.5772	0.2456	0.8228
2.15	$[256, 256]$	noH	2.02	0.5682	0.2586	0.8268
2.15	$[256, 256]$	H8	2.02	0.5885	0.2437	0.8322
2.15	$[128, 512]$	noH	2.03	0.5768	0.2510	0.8278
2.15	$[128, 512]$	H8	2.03	0.5661	0.2531	0.8192
2.15	$[64, 1024]$	noH	2.04	**0.6100**	**0.2300**	**0.8400**
2.15	$[64, 1024]$	H8	2.04	0.5972	0.2380	0.8352
3.15	$[1024, 128, 128]$	noH	3.05	0.7066	0.1703	0.8769
3.15	$[1024, 128, 128]$	H8	3.05	0.6987	0.1774	0.8761
3.15	$[512, 512, 64]$	noH	3.04	0.7009	0.1767	0.8776
3.15	$[512, 512, 64]$	H8	3.04	0.6988	0.1797	0.8785
3.15	$[512, 256, 128]$	noH	3.04	**0.7236**	**0.1587**	0.8823
3.15	$[512, 256, 128]$	H8	3.04	0.6935	0.1806	0.8741
3.15	$[256, 256, 256]$	noH	3.03	0.7076	0.1750	**0.8826**
3.15	$[256, 256, 256]$	H8	3.03	0.7018	0.1788	0.8806
3.15	$[128, 256, 512]$	noH	3.04	0.6987	0.1795	0.8782
3.15	$[128, 256, 512]$	H8	3.04	0.6996	0.1779	0.8776

Профиль 4.15 bpp дополнительно проверялся в полном запуске на всех 18 слоях mid-block. Для него перебирались три распределения кодового бюджета: равномерный профиль $[256, 256, 256, 256]$, профиль $[512, 256, 128, 128]$ и профиль $[512, 512, 128, 128]$. Результаты приведены отдельно.

Преобр.	$K_1, \ldots, K_4$	bpp	Score	LPIPS	CLIP I-I
noH	$[256, 256, 256, 256]$	4.00	0.7801	0.1271	0.9072
noH	$[512, 256, 128, 128]$	3.88	0.7729	0.1341	0.9070
noH	$[512, 512, 128, 128]$	4.00	0.7578	0.1413	0.8992
H8	$[256, 256, 256, 256]$	4.00	0.7850	0.1266	**0.9116**
H8	$[512, 256, 128, 128]$	3.88	0.7403	0.1486	0.8888
H8	$[512, 512, 128, 128]$	4.00	0.7689	0.1373	0.9061
H16	$[256, 256, 256, 256]$	4.00	0.7581	0.1405	0.8986
H16	$[512, 256, 128, 128]$	3.88	0.7521	0.1438	0.8959
H16	$[512, 512, 128, 128]$	4.00	**0.7863**	**0.1247**	0.9110
H32	$[256, 256, 256, 256]$	4.00	0.7787	0.1296	0.9083
H32	$[512, 256, 128, 128]$	3.88	0.7637	0.1408	0.9045
H32	$[512, 512, 128, 128]$	4.00	0.7809	0.1262	0.9071

Результаты показывают, что распределение K по стадиям является одной из наиболее полезных модификаций RVQ. Для профиля 2.15 bpp лучший вариант $[64, 1024]$ без Hadamard

улучшил все основные метрики относительно равномерного $[256, 256]$: Score вырос с 0.5682 до 0.6100, LPIPS уменьшился с 0.2586 до 0.2300, а CLIP I-I вырос с 0.8268 до 0.8400. Для профиля 3.15 bpp лучший вариант $[512, 256, 128]$ также заметно улучшил Score и LPIPS относительно равномерного профиля, а CLIP I-I остался практически на том же уровне. Для профиля 4.15 bpp эффект менее однозначный, но лучший вариант $[512, 512, 128, 128]$ с Hadamard-блоком 16 также оказался сильнее равномерного noH-профиля по Score и LPIPS. Таким образом, перераспределение емкости между стадиями позволяет улучшать качество без грубого выхода за целевой битовый бюджет и является более честной альтернативой простому увеличению K во всех кодбуках.

6.9. Локальный рекодинг после RVQ-стадий

После проверки постадийных профилей была рассмотрена еще одна модификация: локально уточнять выбранные коды после каждой RVQ-стадии. Интуиция у этой идеи следующая. Каждая стадия RVQ аппроксимирует текущий остаток; если после выбора кодов дополнительно перевыбрать индексы уже пройденных стадий, можно было бы получить меньшую ошибку остатка перед переходом к следующему кодбуку.

Однако такая оптимизация может быть слишком жадной. RVQ работает последовательно, и последующие стадии должны иметь возможность исправлять ошибки предыдущих. Если слишком сильно оптимизировать частичный остаток после каждой стадии, можно получить локально хорошее, но глобально менее удачное разложение. Поэтому локальный рекодинг проверялся как отдельная опция, а не включался в метод автоматически.

Эксперимент проводился в том же наборе профилей K при $d = 8$. Финальный full-XTX рекодинг оставался включенным во всех вариантах, а сравнивался только дополнительный локальный шаг после RVQ-стадий. В парных сравнениях локальный рекодинг выиграл только в 2 случаях из 20 и проиграл в 18 случаях. Среднее изменение Score составило -0.0408, среднее изменение LPIPS было $+0.0224$, а среднее изменение CLIP I-I - -0.0185.

Цель	Преобр.	K без	Score без	LPIPS без	CLIP без	K с	Score с	LPIPS с
2.15	noH	$[64, 1024]$	0.6100	0.2300	0.8400	$[256, 256]$	0.5812	0.2466
2.15	H8	$[64, 1024]$	0.5972	0.2380	0.8352	$[256, 256]$	0.5843	0.2441
3.15	noH	$[512, 256, 128]$	0.7236	0.1587	0.8823	$[256, 256, 256]$	0.6688	0.1926
3.15	H8	$[256, 256, 256]$	0.7018	0.1788	0.8806	$[256, 256, 256]$	0.6625	0.1972

Во всех четырех группах лучший вариант без локального рекодинга оказался сильнее лучшего варианта с локальным рекодингом. Особенно заметна деградация на профиле 3.15 bpp: при noH Score снизился с 0.7236 до 0.6688, а LPIPS вырос с 0.1587 до 0.1926. Следовательно, локальный рекодинг после промежуточных RVQ-стадий не имеет практического смысла в текущей реализации. Он ухудшает итоговое качество и поэтому исключается из основной версии метода.

7. Итоговые выводы

В результате экспериментов была выбрана следующая рабочая версия метода: activation-aware RVQ с размерностью фрагмента $d = 8$, распределением размеров кодбуков по RVQ-стадиям, финальным full-XTX рекодингом и без локального рекодинга после промежуточных стадий. Ортогональное преобразование Hadamard не включается безусловно: для каждого битового профиля оно рассматривается как отдельный гиперпараметр, поскольку его эффект зависит от распределения K_m по стадиям.

Главный практический вывод состоит в том, что три модификации действительно оказались полезными. Во-первых, activation-aware функционал позволяет учитывать не только ошибку весов, но и влияние этой ошибки на выходы слоев. Во-вторых, перераспределение размеров кодбуков между RVQ-стадиями улучшает качество без грубого выхода за целевой битовый бюджет. В-третьих, финальный full-XTX рекодинг уточняет дискретные индексы уже после обучения кодбуков и в среднем улучшает внешние метрики, не являясь файнтюнингом модели.

Итоговые настройки RVQ для каждого профиля были выбраны по лучшему Score среди завершенных запусков на всех 18 слоях mid-block. В таблице ниже указана полная битность RVQ с учетом хранения кодбуков, а не только index-bpp.

Целевой профиль	Hadamard	$K_1, \ldots, K_M$	d	bpp	Score
2.15	noH	$[64, 1024]$	8	2.03	0.564
3.15	H32	$[128, 256, 512]$	8	3.02	0.704
4.15	H16	$[512, 512, 128, 128]$	8	4.03	0.786

Итоговое сравнение на всех 18 слоях mid-block приведено ниже. Для каждого профиля показаны четыре лучших базовых метода из предыдущего раздела и выбранный вариант RVQ. Время T_q означает время построения квантизованного представления.

Профиль	Метод	bpp	CLIP I-I	CLIP T-I	LPIPS	Score	T_q, с
2	VQDM	2.02	0.831	0.305	0.244	**0.586**	1857.1
2	RVQ-AW	2.03	0.819	0.305	0.254	0.564	604.2
2	BitsFusion	2.05	0.768	0.312	0.300	0.468	88.4
2	AQUATIC-Diff	2.01	0.760	0.311	0.306	0.453	65.4
2	Vanilla VQ	2.00	0.763	0.311	0.314	0.449	1.02
3	VQDM	3.02	0.885	0.308	0.164	**0.721**	2846.8
3	RVQ-AW	3.02	0.879	0.308	0.175	0.704	992.8
3	Vanilla VQ	3.01	0.828	0.319	0.212	0.617	1.91
3	BitsFusion	3.04	0.829	0.319	0.218	0.611	88.4
3	AQUATIC-Diff	3.02	0.819	0.322	0.229	0.591	66.4
4	VQDM	4.03	0.926	0.310	0.101	**0.825**	3314.0
4	RVQ-AW	4.03	0.911	0.311	0.125	0.786	1400.0
4	Vanilla VQ	4.00	0.881	0.317	0.144	0.737	1.57
4	Q-Diffusion	4.01	0.872	0.322	0.150	0.722	59.3
4	PTQ4DM	4.01	0.876	0.318	0.157	0.719	63.8

По качеству VQDM остается самым сильным методом среди рассмотренных: его AVQ-обучение кодбуков дает лучший Score во всех трех профилях. Однако предложенный RVQ-подход оказывается ближайшим к VQDM и превосходит все остальные базовые методы. Разрыв с VQDM составляет около 0.022 Score на профиле 2 bpp, около 0.017 на профиле 3 bpp и около 0.039 на профиле 4 bpp. При этом отрыв RVQ от более простых базовых методов заметно больше: например, на профиле 3 bpp RVQ получает 0.704 против 0.617 у лучшего базового метода без VQDM.

Время работы также важно для интерпретации результатов. Скалярные методы и простая Vanilla VQ квантуются значительно быстрее, но заметно уступают по качеству. VQDM дает лучшую точность, однако использует более дорогой AVQ-подход с beam search, и эта часть начинает особенно сильно влиять на время при росте числа квантуемых слоев и числа кодбуков. Даже если абсолютные времена отдельных запусков RVQ и VQDM находятся в одном порядке, RVQ принципиально проще масштабируется: он не выполняет совместный beam search по комбинациям кодов, а последовательно аппроксимирует остаток. Поэтому предложенный метод занимает промежуточное положение: он существенно качественнее легких базовых методов и ближе всех к VQDM, но имеет меньшую вычислительную стоимость, чем полноценный AVQ.

Список литературы

[1] Denoising Diffusion Probabilistic Models
https://arxiv.org/abs/2006.11239

[2] High-Resolution Image Synthesis with Latent Diffusion Models
https://arxiv.org/abs/2112.10752

[3] Stable Diffusion v1-5 Model Card
https://huggingface.co/runwayml/stable-diffusion-v1-5

[4] Hugging Face Diffusion Models Course
https://huggingface.co/learn/diffusion-course

[5] Post-training Quantization on Diffusion Models
https://arxiv.org/abs/2211.15736

[6] Q-Diffusion: Quantizing Diffusion Models
https://arxiv.org/abs/2302.04304

[7] BitsFusion: 1.99 bits Weight Quantization of Diffusion Model
https://arxiv.org/abs/2406.04333

[8] AQUATIC-Diff: Additive Quantization for Truly Tiny Compressed Diffusion Models
https://arxiv.org/abs/2506.05960

[9] Accurate Compression of Text-to-Image Diffusion Models via Vector Quantization
https://arxiv.org/abs/2409.00492

[10] Vector Quantization and Signal Compression
https://link.springer.com/book/9780792391814